

AURIX's Developer Role Guide

Role : Core Engine Architect (AI & Sandbox)

Assigned to: DIVYANSH

1. The "Why" of Your Component

You own everything that happens after the raw scan data hits the Redis queue. You are building the "Muscle and the Brain" of AURIX. Without your component, AURIX is just a standard, noisy vulnerability scanner. Your job is to take raw, unverified scan data, use AI to reason about it, write actual exploit code to test it, and generate a verified patch in a secure sandbox.

2. Your Domain in the Architecture (HLD & LLD References)

As the Core Engine Architect, you are responsible for specific layers within the High-Level and Low-Level Designs:

- **HLD Ownership:** You own **Layer 3 (Dedicated Worker Node)**, **Layer 4 (LangGraph Agentic Orchestrator)**, and **Layer 5 (Isolated Execution)**. You are responsible for the Task Worker Daemon, the Scan Engine, the Triage Node, and the entire AI ecosystem.
- **LLD Ownership (The Brain):** You must implement the `AurixState` Python TypedDict exactly as defined in the **Defining LangGraph State Schema** document. This state acts as the shared memory object passed between all LangGraph nodes.
- **LLD Ownership (The Flow):** You must program the execution loop according to the **LangGraph State Machine Orchestrator-LLD** diagram. This includes building the Consensus Gate that checks if the `confidence_score` is greater than 0.80 before triggering the Red and Blue agents.

3. Step-by-Step Guide

1. **Build the Triage Node & Semantic Slicer:** Write the Python script that pulls the next job from Redis, unzips the payload, and parses the initial finding. Instead of just grabbing 50 lines of code, implement an AST (Abstract Syntax Tree) or semantic slicer to trace the execution path across multiple files, feeding a rich "Semantic Slice" to the AI.
2. **RAG Threat Intelligence :** Before evaluation, perform a quick similarity search against a local Vector DB (like ChromaDB) of OWASP guidelines and past CVEs to ground the AI's reasoning in factual security data.
3. **Logic Agent & Consensus Gate:** Prompt the Logic Agent to evaluate the slice and output

a strict `confidence_score`. Write the conditional edge in LangGraph: If `confidence_score > 0.80`, route to the Red Agent. If not, drop the finding and reset the state.

4. **Red Agent & Blue Agent Generation:** Prompt the Red Agent to write a Python `poc_script.py` tailored exactly to the vulnerability. Simultaneously or sequentially, prompt the Blue Agent to write the `patch_code`.
5. **Ephemeral Docker Sandbox :** Use the Docker Python SDK to dynamically spin up a container with `--network none` (strict network isolation). Execute the Red Agent's script against the mocked environment. Ensure the container is destroyed (`auto_remove=True`) the millisecond the test finishes to prevent any host breaches.
6. **The "Reflexion" Loop:** If the Docker sandbox throws a syntax or execution error (e.g., `IndentationError`), capture `stderr`. Route the state *back* to the Red Agent with the error logs attached to self-correct the script. Cap this at 3 retries.
7. **Parallel Wargaming :** Utilize LangGraph's async capabilities (Send API) to fan out the state machine. Process multiple vulnerabilities from the same repository simultaneously in parallel Docker containers to drastically reduce scan times.
8. **Compile & Webhook:** Once the state reaches the end, map the results to the final JSON payload and POST it to Bhavya's Webhook URL.

4. State Pipeline & Node Output Flow (The Graph Blueprint)

To ensure the LangGraph state machine operates flawlessly, you must map the exact outputs of each node and their downstream destinations:

- **Node 1 (Triage & Context Slicer):**
 - **Generates:** `target_vulnerabilities` (List of dictionaries) and `current_vuln` (Dict of the active bug being processed).
 - **Forwards to:** The queue loop, passing the first `current_vuln` to Node 2.
- **Node 2 (RAG Context Injector):**
 - **Generates:** `threat_intel` (String/Markdown of matched OWASP methodologies).
 - **Forwards to:** Node 3 (Logic Agent).
- **Node 3 (Logic Agent):**
 - **Generates:** `confidence_score` (Float) and `logic_reasoning` (String).
 - **Forwards to:** The Consensus Gate (Conditional Edge). If `< 0.80`, state clears and returns to Node 1 for the next item. If `>= 0.80`, forwards to Node 4 and Node 5.
- **Node 4 (Red Agent):**
 - **Generates:** `poc_script` (String formatted as runnable Python code).
 - **Forwards to:** Node 6 (Sandbox).
- **Node 5 (Blue Agent):**
 - **Generates:** `patch_code` (String formatted as replacement code).
 - **Forwards to:** The final State Reducer (Node 7).
- **Node 6 (Docker Sandbox):**
 - **Generates:** `wargame_status` (Boolean) and `sandbox_logs` (String containing `stderr/stdout`).

- **Forwards to:** Reflexion Conditional Edge. If failed and retries < 3, forwards back to Node 4. If passed (or out of retries), forwards to Node 7.
- **Node 7 (State Reducer):**
 - **Generates:** The finalized dictionary object representing the fully verified bug.
 - **Forwards to:** The `verified_reports` state variable via the `append_to_list` reducer function.

5. Expected Output / Streamlining Flow

- **Your engine must output a single, finalized list of verified reports.**
- As the system loops, every time a bug survives the Docker sandbox, the final report is appended to the `verified_reports` list using the `append_to_list` reducer.
- When the `target_vulnerabilities` queue is entirely empty, you must return this fully compiled JSON payload to the Backend API via the Webhook, completing the scan cycle autonomously.

6. Critical "Do Not Miss" Constraints

- **Strict Schema Enforcement:** The LLMs must output strictly structured JSON. You must use Pydantic models with LangChain's `.with_structured_output()` to guarantee the `confidence_score` returns as a float and the `is_exploitable` returns as a boolean.
- **Sandbox Security:** Do not cause destructive system harm; the PoC script must focus purely on proving the specific finding. Containers must be completely ephemeral and isolated from the host network.
- **Clear State Reset:** After a finding is processed (whether it fails or succeeds), you must clear the `current_vuln` and `sandbox_logs` state to drop the finding before moving to the next item in the queue.

7. Prerequisites & Skills Needed

- Python, LangChain, Docker SDK, OpenAI/Anthropic APIs.
- Deep understanding of LangGraph state management (TypedDict, Async workflows, Reducers) and LLM orchestration.
- Abstract Syntax Trees (AST) or static analysis basics for the Semantic Slicer.

8. Reference Materials

Docs provided by tech leads

- [aurix-HLD.mmd](#) and [aurix-HLD.png](#)
- [LangGraph State Machine Orchestrator-LLD.mmd](#) and [png](#)
- [Defining LangGraph State Schema-advisory.docx](#),
- [System Prompts\(Logic Agent, Red Agent,Blue Agent\)-advisory.docx](#).

External documentation

- [LangGraph Documentation](#)
- [Docker Python SDK Docs](#)
- [OpenAI Structured Outputs Guide](#)

9. Component Build List & Utility

- **Triage Node & Semantic Slicer:** Feeds highly accurate, multi-file code context to the AI.
- **RAG Context Injector:** Grounds the AI in real-world threat intelligence.
- **AurixState & LangGraph Orchestrator:** The core decision-making loop.
- **Consensus Gate:** The strict >0.80 probability filter that saves compute time.
- **Reflexion Engine:** Allows the AI to fix its own broken exploit scripts.
- **Ephemeral Docker Runner:** The highly secure, auto-destructing testing environment.

9. 🚨 CRITICAL DEPENDENCY (Blocker for Others)

You block the Backend Engineer (BHAVYA). Before you write any complex AI code, you and the Backend Engineer must agree on the exact JSON format of the final `verified_vulnerabilities` payload. If they don't know what you are sending, they cannot correctly map it to the database table defined in the ERD.